

Amazon Networking Services

In the previous chapters, we learned about Amazon compute and storage services, focusing on EC2, EBS, EFS, and S3. We provisioned EC2 instances and the cloud storage that can be used by EC2 and other services. In this chapter, we will discuss Amazon cloud networking, where all the cloud resources are connected and communicate with each other.

We will cover the following topics in this chapter:

- Computer network basics – the fundamental network concepts, such as IP address, network address, and CIDR notation.
- Amazon **Virtual Private Cloud (VPC)** – the Amazon cloud network where EC2 and other services communicate with each other. We will provision VPCs, subnets, and EC2 instances in the VPC/subnets and explore EC2 instance communications within the VPC and to the internet.
- AWS network security – this becomes a priority once our VPC and EC2 instances are exposed to the internet. We will show how to build and configure network firewalls for cloud resource protection.
- AWS Direct Connect – the information highway connecting your private on-premises data centers to the AWS public cloud to form a hybrid cloud.
- The Amazon cloud **Domain Name System (DNS)** – Route 53 is a reliable and scalable DNS service that provides essential functionality for hosting and managing your domains and resources in the AWS cloud.
- The Amazon **Content Delivery Network (CDN)** – CloudFront is an AWS CDN service that delivers content, such as web pages, videos, and images, to users with low latency and high speeds.

Cloud networking is one of the most important AWS services. By learning about the basic cloud networking concepts and practicing the fundamental skills in this chapter, you will build a solid and strong foundation for constructing your AWS cloud framework. We will start the chapter by reviewing computer network basics.

Reviewing computer network basics

In *Chapter 1*, we learned that a computer network is a collection of interconnected devices, such as computers, servers, printers, and other devices, that are linked together to share resources and communicate with each other. A subnet, or subnetwork, is a smaller network that is created by partitioning a larger network into smaller segments, typically used to improve network performance, security, and management. Each subnet is identified by a unique network address, and devices within the same subnet share the same network address prefix. Each device has a unique address in the network/subnet. For a computer to talk to another one in a computer network, it must have an address for communication, which we call an **IP address**.

IP address

An **IP address**, or an **Internet Protocol** address, is a numerical address assigned to a device in a computer network for communication with the other devices that are also assigned IP addresses. An IP address serves as an identifier for the device on the network and allows it to send and receive data. There are two versions of IP addresses in use today: IPv4 and IPv6. IPv4 addresses consist of four decimal numbers separated by dots, while IPv6 addresses use a hexadecimal format with eight sets of four characters separated by colons. We will mainly use IPv4 in our book. *Figure 3.1* shows a fictitious website, `www.zeebestbuy.com`, and its IP address

zeebestbuy.com
172.217.9.36

10101100. 11011001. 00001001.00100100

Figure 3.1 – An IP address

As we can see, an IPv4 address can be represented by four decimal numbers, 172 . 217 . 9 . 36, or by 4 bytes (32 bits) of binary numbers, 10101100. 11011001. 00001001.00100100. Since an IP address has 4 bytes or 32 bits, the total number of addresses in this IP space is 2^{32} . We cannot put this number of computers in one network, so we need to develop a way to separate the network into subnets, which requires something called **Classless Inter-Domain Routing (CIDR)**.

CIDR

CIDR is a method used to allocate IP addresses in networks and specify network addresses. In the CIDR notation, a network address is represented by an IP address followed by a slash (/) and a number indicating the number of bits in the network address. For example, the CIDR notation 172 . 217 . 9 . 0 / 24 represents a network address – the first 24 bits (10101100. 11011001. 00001001) represent the network portion and are *fixed*, and the remaining 8 bits represent the host portion and are *varied* (from 00000000 to 11111111). For the subnet 172 . 217 . 9 . 0 / 24, since 24 bits out of 32 are fixed and 8 bits are variable, there are $2^8=256$ IP addresses in the subnet:

172.217.9.0 to 172.217.9.255. Figure 3.2 shows CIDR notations for some subnets, from /32 to /24, and the available IP addresses for the subnets:

Subnet Mask Hierarchy

Subnet Mask	CIDR	Binary Notation	Available Addresses Per Subnet
255.255.255.255	/32	11111111.11111111.11111111.11111111	1
255.255.255.254	/31	11111111.11111111.11111111.11111110	2
255.255.255.252	/30	11111111.11111111.11111111.11111100	4
255.255.255.248	/29	11111111.11111111.11111111.11111000	8
255.255.255.240	/28	11111111.11111111.11111111.11110000	16
255.255.255.224	/27	11111111.11111111.11111111.11100000	32
255.255.255.192	/26	11111111.11111111.11111111.11000000	64
255.255.255.128	/25	11111111.11111111.11111111.10000000	128
255.255.255.0	/24	11111111.11111111.11111111.00000000	256
255.255.254.0	/23	11111111.11111111.11111110.00000000	512
255.255.252.0	/22	11111111.11111111.11111100.00000000	1024
255.255.248.0	/21	11111111.11111111.11111000.00000000	2048
255.255.240.0	/20	11111111.11111111.11110000.00000000	4096
255.255.224.0	/19	11111111.11111111.11100000.00000000	8192
255.255.192.0	/18	11111111.11111111.11000000.00000000	16384
255.255.128.0	/17	11111111.11111111.10000000.00000000	32768
255.255.0.0	/16	11111111.11111111.00000000.00000000	65536
255.254.0.0	/15	11111111.11111110.00000000.00000000	131072
255.252.0.0	/14	11111111.11111100.00000000.00000000	262144
255.248.0.0	/13	11111111.11111000.00000000.00000000	524288
255.240.0.0	/12	11111111.11110000.00000000.00000000	1048576
255.224.0.0	/11	11111111.11100000.00000000.00000000	2097152
255.192.0.0	/10	11111111.11000000.00000000.00000000	4194304
255.128.0.0	/9	11111111.10000000.00000000.00000000	8388608
255.0.0.0	/8	11111111.00000000.00000000.00000000	16777216

Figure 3.2 – CIDR notation

With IP address and network address notations, we can construct any networks and connect them with routing devices. And that's how we build the internet, which will be discussed next.

The internet

The internet is a global network of interconnected computer networks that allows the exchange of information and communication among users around the world. It is composed of millions of devices, including computers and networking hardware devices, and software applications, protocols, and standards that enable these devices to communicate and share data with each other.

At a high level, the internet is made up of physical infrastructure, such as computers, routers, and switches. A router is a network device that connects multiple networks and routes data packets between them. Routers use a route table to determine the best path for data packets to take between networks, and they can be used to filter and direct traffic based on various criteria, such as port numbers, IP addresses, and protocols. Each subnet must be associated with one and only one route table. A switch

is a network device that connects multiple devices within a network. Switches use MAC addresses to direct traffic to the correct device and can be used to segment networks and improve performance by limiting the amount of traffic that flows through each segment.

On the internet, routers, switches, and computers work together, under network routing protocols such as **Transmission Control Protocol (TCP)** and IP. Just like computers have evolved from physical machines to virtual machines, networks have evolved from hardware-based physical networks to software-based virtual networks. In the next section, we will discuss Amazon VPC, which is a virtual network in the cloud.

Understanding Amazon Virtual Private Cloud

A VPC is a network in the Amazon cloud. With Amazon VPC, you can assign custom IP address ranges, create subnets, and configure route tables, network gateways, and security settings. You can launch instances in a specified subnet of a VPC. By default, you can create five VPCs per AWS account per region, and you can increase this default quota by contacting Amazon cloud support. AWS VPC provides several benefits over traditional data center-based network designs, including the following:

- **High availability:** Amazon VPC is software based. Within a VPC, the routers are virtual and provide high availability. VPCs are regional in AWS and thus a VPC can span multiple AZs, with each subnet mapping to an AZ.
- **Flexibility:** Amazon VPC provides the flexibility to design and build your network topology to meet your specific business and technical requirements. With Amazon VPC, you select the IP address range for VPCs and subnets, configure route tables, and associate them with subnets.
- **Isolation:** Amazon VPC provides isolation of cloud resources, allowing you to create separate virtual networks for different applications or workloads, for example, separate a production VPC from a development VPC. A VPC is logically isolated from other VPCs in the same AWS account.
- **Time and cost effectiveness:** Amazon VPCs are created and used at no cost. Since they are virtual, you can provision VPCs and subnets on the fly and reduce the cost of your network infrastructure.
- **Security:** In the Amazon cloud, you can define **Network Access Control Lists (NACLs)** to control traffic in and out of subnets.
- **Connectivity to on-premises:** You can configure Amazon VPC to connect the virtual network to an on-premises data center using a VPN or AWS Direct Connect. This allows enterprises to extend their existing IT infrastructure to the cloud and access AWS services securely.

Next, we will provision our VPC networks in the Amazon cloud. We will use the AWS cloud console to show how to create a VPC and subnet. We will use CloudShell to create an AWS cloud, including a VPC, subnets, and instances, and then configure the VPC/subnet/EC2 instances for internet access. Let's start with the first part of building our AWS cloud framework.

Part one – creating a VPC with subnets

In the first part, we will use the AWS console to create a VPC and a subnet under the VPC, to demonstrate how to use the AWS console in creating networks in the cloud.

Creating a VPC and a subnet using the AWS cloud console

Log in to the AWS console, search for VPC services, and select **Create VPC**, then fill in the VPC settings. Enter `my vpc1` for the name. For simplicity, we will select **No IPv6 CIDR block** and choose a CIDR of `10.10.0.0/16` for our VPC. Click **Create VPC**, as shown in *Figure 3.3*:

The screenshot displays the 'VPC settings' page in the AWS console. It is divided into two main sections: 'VPC settings' and 'Tags'.

VPC settings

- Resources to create**: Two radio buttons are present. 'VPC only' is selected, while 'VPC and more' is unselected.
- Name tag - optional**: A text input field contains the value 'my VPC1'.
- IPv4 CIDR block**: Two radio buttons are present. 'IPv4 CIDR manual input' is selected, while 'IPAM-allocated IPv4 CIDR block' is unselected.
- IPv4 CIDR**: A text input field contains the value '10.10.0.0/16'.
- IPv6 CIDR block**: Four radio buttons are present. 'No IPv6 CIDR block' is selected, while 'IPAM-allocated IPv6 CIDR block', 'Amazon-provided IPv6 CIDR block', and 'IPv6 CIDR owned by me' are unselected.
- Tenancy**: A dropdown menu is set to 'Default'.

Tags

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key	Value - optional	
Q Name	Q my VPC1	Remove

Add new tag

You can add 49 more tags.

At the bottom right, there are two buttons: 'Cancel' and 'Create VPC'.

Figure 3.3 – VPC settings

After the VPC is created, go back to the VPC dashboard and then to **Subnets**, then click **Create subnet**. Fill in the subnet settings; choose **my VPC1** as the VPC, enter `public-subnet` as the name, set its CIDR to `10.10.1.0/24`, and click **Create subnet**, as shown in *Figure 3.4*:

VPC ID
Create subnets in this VPC.

vpc-0194dc489edcfb09b (my VPC1) ▼

Associated VPC CIDRs

IPv4 CIDRs

10.10.0.0/16

Subnet settings
Specify the CIDR blocks and Availability Zone for the subnet.

Subnet 1 of 1

Subnet name
Create a tag with a key of 'Name' and a value that you specify.

public-subnet

The name can be up to 256 characters long.

Availability Zone [Info](#)
Choose the zone in which your subnet will reside, or let Amazon choose one for you.

US East (N. Virginia) / us-east-1a ▼

IPv4 CIDR block [Info](#)

Q 10.10.1.0/24 X

▼ **Tags - optional**

Key	Value - optional	
Q Name X	Q public-subnet X	Remove

Add new tag

You can add 49 more tags.

Remove

Add new subnet

Cancel **Create subnet**

Figure 3.4 – Subnet settings

Note

As we discussed earlier, a VPC with an IPv4 CIDR block of `10.10.0.0/16` has 65,536 total IP addresses and can be divided into subnets. For each subnet, there are some addresses reserved for Amazon. For example, the subnet `10.10.1.0/24` has a total of 256 total IP addresses, but the following 5 IP addresses are reserved, thus leaving 251 IP addresses available to create EC2 instances in the subnet:

`10.0.1.0`

Network address

`10.0.1.1`

Internal communication

`10.0.1.2`

DNS resolution

`10.0.1.3`

Future use

`10.0.1.255`

Network broadcast address

Creating an AWS cloud network using CloudShell

As we discussed earlier, we can also use AWS CloudShell to create and manage any cloud resources with the **Command Line Interface (CLI)**. In this section, we will use CloudShell to recreate a VPC named **VPC1** and two subnets named **public-subnet** and **private-subnet** under VPC1, in the US-west-2 region, and launch two instances – one Linux instance in **public-subnet** with a public IP address and another Linux instance in **private-subnet** without a public IP address:

1. Create VPC1:

```
aws ec2 create-vpc --cidr-block
10.10.0.0/16 --tag-specifications
ResourceType=vpc,Tags='[{Key=Name,Value="VPC1"}]'
```

```
[cloudshell-user@ip-10-2-126-119 ~]$ aws ec2 create-vpc \  
> --cidr-block 10.10.0.0/16 \  
> --tag-specifications ResourceType=vpc,Tags='[{Key=Name,Value="VPC1"}]' \  
{  
  "Vpc": {  
    "CidrBlock": "10.10.0.0/16",  
    "DhcpOptionsId": "dopt-f8579e9d",  
    "State": "pending",  
    "VpcId": "vpc-0e6043d6091faa7cd",  
    "OwnerId": "317332158300",  
    "InstanceTenancy": "default",  
    "Ipv6CidrBlockAssociationSet": [],  
    "CidrBlockAssociationSet": [  
      {  
        "AssociationId": "vpc-cidr-assoc-04116e8e5e98b256f",  
        "CidrBlock": "10.10.0.0/16",  
        "CidrBlockState": {  
          "State": "associated"  
        }  
      }  
    ],  
    "IsDefault": false,  
    "Tags": [  
      {  
        "Key": "Name",  
        "Value": "VPC1"  
      }  
    ]  
  }  
}
```

Figure 3.5 – Create VPC1

2. **Create public-subnet and private-subnet in VPC1**, as shown in *Figure 3.6*:

```
aws ec2 create-subnet --vpc-id vpc-0e6043d6091faa7cd \  
--cidr-block 10.10.1.0/24 --tag-specifications ResourceType=subn  
et,Tags='[{Key=Name,Value="public-subnet"}]' \  
aws ec2 create-subnet --vpc-id vpc-0e6043d6091faa7cd --cidr-  
block 10.10.2.0/24 --tag-specifications ResourceType=subnet,Tags  
='[{Key=Name,Value="private-subnet"}]'
```



```
[cloudshell-user@ip-10-2-126-119 ~]$ aws ec2 create-subnet \
> --vpc-id vpc-0e6043d6091faa7cd \
> --cidr-block 10.10.2.0/24 \
> --tag-specifications ResourceType=subnet,Tags='[{"Key=Name,Value="private-subnet"}]'
```

```
{
  "Subnet": {
    "AvailabilityZone": "us-west-2a",
    "AvailabilityZoneId": "usw2-az1",
    "AvailableIpAddressCount": 251,
    "CidrBlock": "10.10.2.0/24",
    "DefaultForAz": false,
    "MapPublicIpOnLaunch": false,
    "State": "available",
    "SubnetId": "subnet-0328a7940791d9fc0",
    "VpcId": "vpc-0e6043d6091faa7cd",
    "OwnerId": "317332158300",
    "AssignIpv6AddressOnCreation": false,
    "Ipv6CidrBlockAssociationSet": [],
    "Tags": [
      {
        "Key": "Name",
        "Value": "private-subnet"
      }
    ],
    "SubnetArn": "arn:aws:ec2:us-west-2:317332158300:subnet/subnet-0328a7940791d9fc0",
    "EnableDns64": false,
    "Ipv6Native": false,
    "PrivateDnsNameOptionsOnLaunch": {
      "HostnameType": "ip-name",
      "EnableResourceNameDnsARecord": false,
      "EnableResourceNameDnsAAAARecord": false
    }
  }
}
```

Figure 3.6 – Create subnets

3. **Create two EC2 instances**, one in each subnet. EC2-1 is in the public subnet and is assigned a public IP address, while EC2-2 is in the private subnet and does not have a public IP address. The commands are as follows and a screenshot is provided in *Figure 3.7*:

```
aws ec2 run-instances --image-id ami-0ef0b498cd3fe129c --count 1 --instance-type t2.micro --subnet-id subnet-0871b6aaa7092c075 --key-name mywestkp --region us-west-2 --associate-public-ip-address
aws ec2 run-instances --image-id ami-0ef0b498cd3fe129c --count 1 --instance-type t2.micro --subnet-id subnet-0328a7940791d9fc0 --key-name mywestkp --region us-west-2 --no-associate-public-ip-address
```

```
[cloudshell-user@ip-10-6-43-95 ~]$ aws ec2 run-instances --image-id ami-0ef0b498cd3fe129c --count 1 \
> --instance-type t2.micro --subnet-id subnet-0328a7940791d9fc0 \
> --key-name mywestkp --region us-west-2 --no-associate-public-ip-address
{
  "Groups": [],
  "Instances": [
    {
      "AmiLaunchIndex": 0,
      "ImageId": "ami-0ef0b498cd3fe129c",
      "InstanceId": "i-0288c5d94570afe62",
      "InstanceType": "t2.micro",
      "KeyName": "mywestkp",
      "LaunchTime": "2023-02-14T01:56:33+00:00",
      "Monitoring": {
        "State": "disabled"
      },
      "Placement": {
        "AvailabilityZone": "us-west-2a",
        "GroupName": "",
        "Tenancy": "default"
      },
      "PrivateDnsName": "ip-10-10-2-11.us-west-2.compute.internal",
      "PrivateIpAddress": "10.10.2.11",
    }
  ]
}
```

Figure 3.7 – Create instances

Figure 3.8 shows a network diagram of the VPC we have built so far, including VPC1, its two subnets, and the two instances – EC2-1 in the public subnet and EC2-2 in the private subnet:

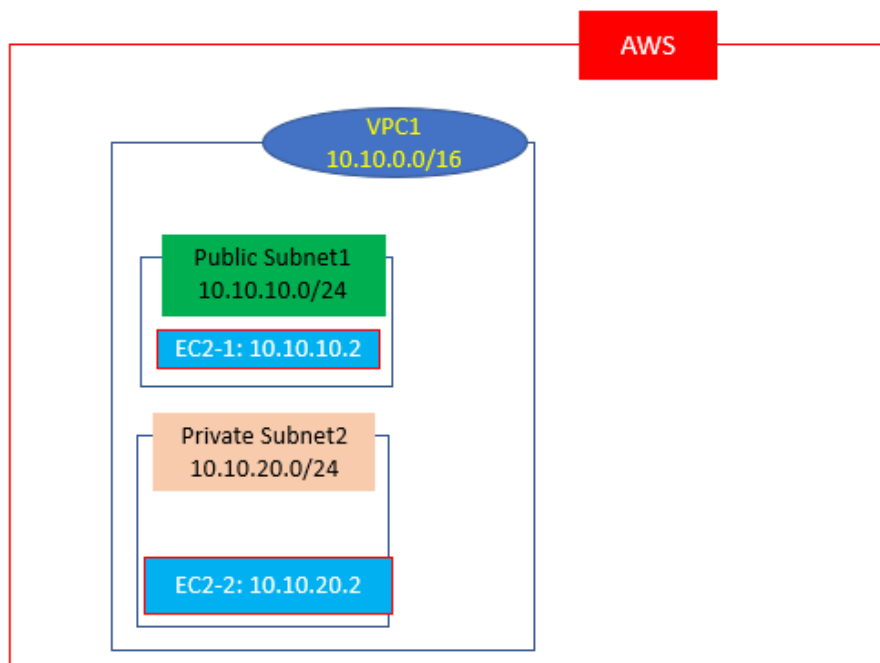


Figure 3.8 – VPC network

Now that we have provisioned VPCs/subnets/EC2 instances, it's time to discuss how to access them from the internet.

Accessing the EC2 instances from the internet

While VPCs are isolated networks in the cloud, there are needs to access an EC2 from the internet and for the EC2 instances to access the internet, such as to patch the guest OS. For the EC2-1 instance in public subnet1 in VPC1 to communicate with the internet, we need to open a route from the public subnet to the internet, with three steps:

1. **Create IGW1 and attach it to VPC1**

To open a route from our VPC to the internet, we need to create an **Internet Gateway (IGW)** and attach it to VPC1.

From the AWS console, go to the VPC dashboard and find the **Internet Gateway** section. Click **Create internet gateway**, and in the new window, provide a name tag for the IGW. We will call it IGW1, as shown in *Figure 3.9*:

VPC > Internet gateways > Create internet gateway

Create internet gateway [Info](#)

An internet gateway is a virtual router that connects a VPC to the internet. To create a new internet gateway specify the name for the gateway below.

Internet gateway settings

Name tag
Creates a tag with a key of 'Name' and a value that you specify.

Tags - optional

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key	Value - optional	
Name	IGW1	Remove

You can add 49 more tags.

Figure 3.9 – Create IGW1

Attach IGW1 to VPC1, as shown in *Figure 3.10*.

The screenshot shows the 'Attach to VPC (igw-02afe3e9778325c7e)' page in the AWS Management Console. The breadcrumb trail is 'VPC > Internet gateways > Attach to VPC (igw-02afe3e9778325c7e)'. The main heading is 'Attach to VPC (igw-02afe3e9778325c7e)' with an 'Info' link. Below this, there's a section titled 'VPC' with the instruction: 'Attach an internet gateway to a VPC to enable the VPC to communicate with the internet. Specify the VPC to attach below.' Underneath, 'Available VPCs' are listed with the instruction 'Attach the internet gateway to this VPC.' A search bar contains 'vpc-0e6043d6091faa7cd'. At the bottom, there's a section for 'AWS Command Line Interface command' and two buttons: 'Cancel' and 'Attach internet gateway'.

Figure 3.10 – Attach IGW1 to VPC1

2. Add an internet route to public subnet1

Now we need to modify the route table for VPC1's public subnet1 and add a route to the internet – via an IGW. Note that when we created VPC1 and its two subnets earlier, there was an *invisible* VPC router created to route the traffic within VPC1; all subnets with a VPC are routable. We will now check out the route table for VPC1, as shown in *Figure 3.11*:

The screenshot shows the 'Route table' page for 'rtb-0398b091a7c399a0e / VPC1-RT'. The breadcrumb trail is 'VPC > Route tables > rtb-0398b091a7c399a0e'. The title is 'rtb-0398b091a7c399a0e / VPC1-RT' with an 'Actions' dropdown. A blue banner at the top says 'You can now check network connectivity with Reachability Analyzer' with a 'Run Reachability Analyzer' button. Below this is the 'Details' section with a table of metadata:

Route table ID rtb-0398b091a7c399a0e	Main Yes	Explicit subnet associations -	Edge associations -
VPC vpc-0e6043d6091faa7cd VPC1	Owner ID 317332158300		

Below the details are tabs for 'Routes', 'Subnet associations', 'Edge associations', 'Route propagation', and 'Tags'. The 'Routes' tab is selected, showing 'Routes (1)'. There's a search bar 'Filter routes' and a dropdown 'Both'. A table lists the routes:

Destination	Target	Status	Propagated
10.10.0.0/16	local	Active	No

Figure 3.11 – Route table for VPC1

Click **Edit routes** and add a new route with a destination of `0.0.0.0/0` (meaning anywhere) and a target of IGW1, which is the IGW we created and attached to VPC1 earlier. Click **Save changes**, as shown in *Figure 3.12*:

VPC > Route tables > rtb-0398b091a7c399a0e > Edit routes

Edit routes

Destination	Target	Status	Propagated
10.10.0.0/16	Q local X	Active	No
Q 0.0.0.0/0 X	Q igw-02afe3e9778325c7e X	-	No

Add route

Cancel Preview **Save changes**

Figure 3.12 – Add internet route to route table

Go back to the route table and then to **Subnet association | Edit subnet associations** and associate the route table with the public subnet we created earlier, as shown in *Figure 3.13*:

VPC > Route tables > rtb-0398b091a7c399a0e > Edit subnet associations

Edit subnet associations

Change which subnets are associated with this route table.

Available subnets (1/2)

Q Filter subnet associations

	Name	Subnet ID	IPv4 CIDR	IPv6 CIDR	Route table ID
☒	public-subnet	subnet-0871b6aaa7092c075	10.10.1.0/24	-	Main (rtb-0398b091a7c399a0e / VPC1-RT)
☐	private-subnet	subnet-0328a7940791d9fc0	10.10.2.0/24	-	Main (rtb-0398b091a7c399a0e / VPC1-RT)

Selected subnets

subnet-0871b6aaa7092c075 / public-subnet X

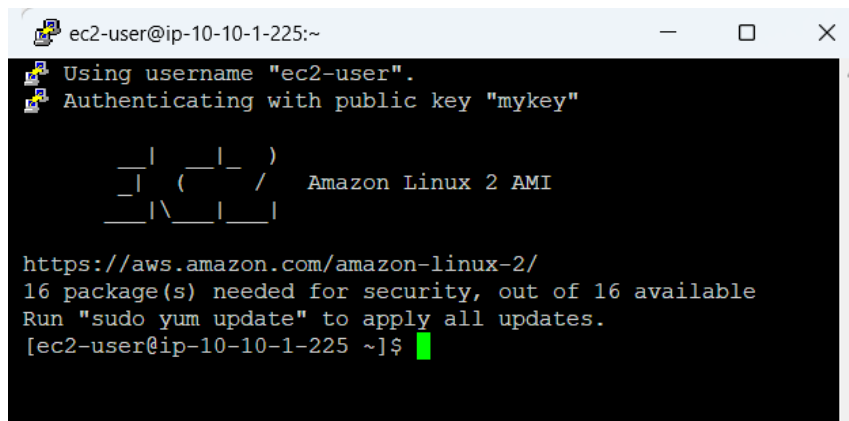
Cancel **Save associations**

Figure 3.13 – Associate route table to subnet

Click **Save associations**. Now, we have built a route from VPC1 to the internet, so all internet traffic will go to IGW1 as the next hop.

3. SSH to the EC2 instance in the public subnet

In *Chapter 1*, we have introduced PuTTY to SSH into the Linux EC2 instances in the cloud. Now, we SSH through the internet into the EC2-1 instance, as shown in *Figure 3.14*:

A terminal window titled 'ec2-user@ip-10-10-1-225:~' showing the process of SSHing into an EC2 instance. It displays the username 'ec2-user', authentication with a public key 'mykey', the Amazon Linux 2 AMI logo, a link to the AWS Linux 2 documentation, and a message about security updates. The prompt is '[ec2-user@ip-10-10-1-225 ~]\$' with a green cursor.

```
ec2-user@ip-10-10-1-225:~  
Using username "ec2-user".  
Authenticating with public key "mykey"  
  
  _ | _ | _ )  
  _ | ( _ | /  Amazon Linux 2 AMI  
  _ | \ _ | _ |  
  
https://aws.amazon.com/amazon-linux-2/  
16 package(s) needed for security, out of 16 available  
Run "sudo yum update" to apply all updates.  
[ec2-user@ip-10-10-1-225 ~]$
```

Figure 3.14 – SSH to EC2-1 from the internet

We have set up the route between the EC2-1 instance and the internet. *Figure 3.15* shows the AWS network we have built so far:

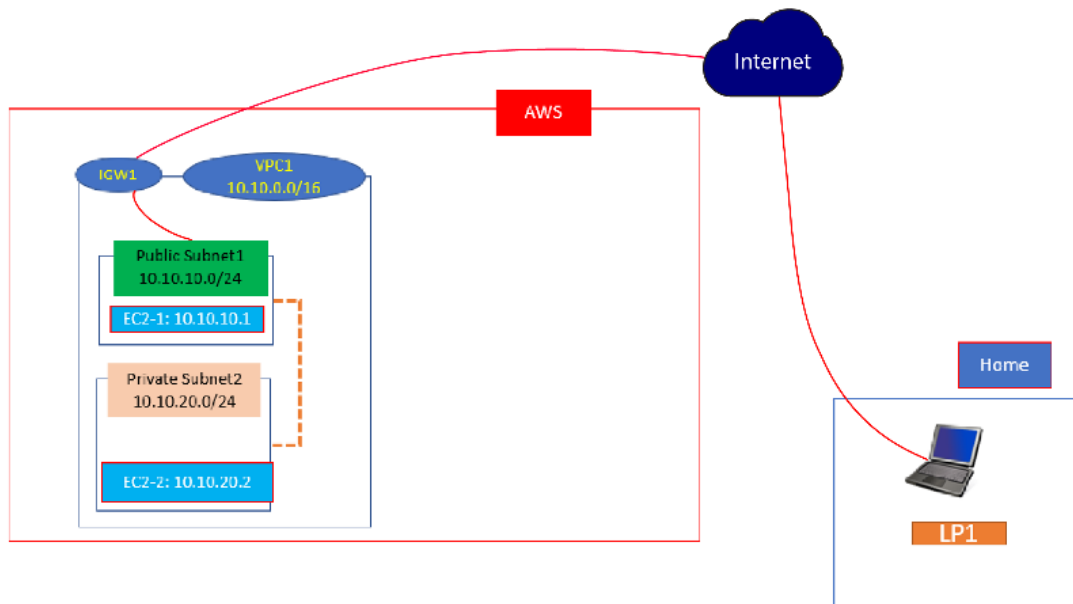


Figure 3.15 – AWS cloud with internet access

Now, let's add more VPC/subnet/EC2 instances to the cloud.

Part two – Provisioning more cloud resources and connecting them together

In this part, we will add more VPCs, subnets, and EC2 instances into the cloud and connect them together by peering the VPCs.

Provisioning more cloud resources

With the same procedure, let us create VPC2, subnet8 in VPC2, and an EC2 instance in subnet8. The network architecture is shown in *Figure 3.16*. Note EC2-2 and EC2-8 do not have public IPs assigned:

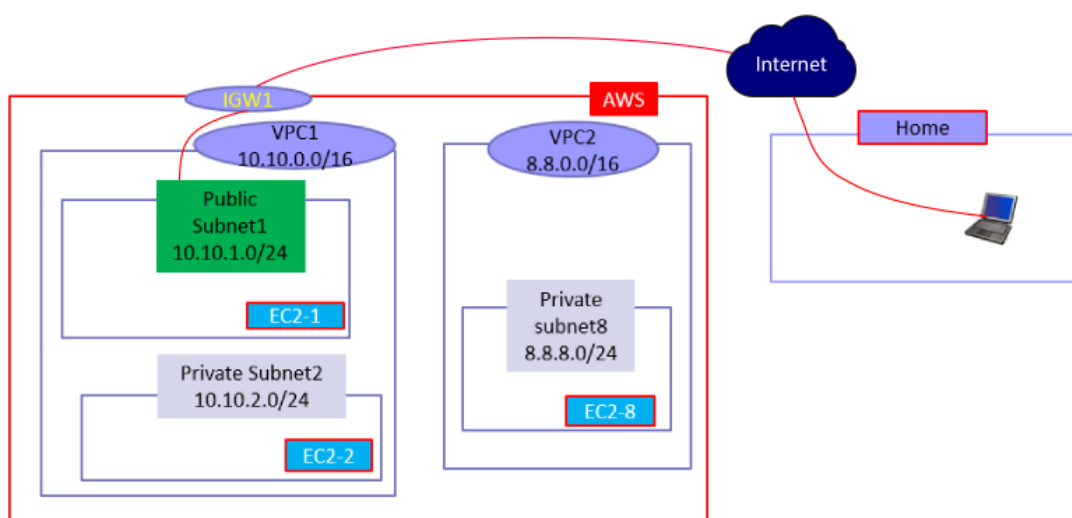


Figure 3.16 – VPC network architecture

As we addressed earlier, all EC2 instances in the same VPC can communicate with each other. From EC2-1, we can ping EC2-2 after we have configured EC2-2's SG to allow ICMP (ping) from EC2-1. However, we cannot ping EC2-8 from EC2-1, since there are no routes between them. Let's resolve this issue by *peering* VPC1 and VPC2 together.

Peering the VPCs

In the Amazon cloud, we can peer two VPCs in the same AWS account, no matter which region they sit in, and we can also peer two VPCs in different AWS accounts, provided they agree on peering with each other. The following figure provides an example of peering two VPCs in the same account. From the AWS console, go to the VPC dashboard and then click **Peering connections | Create peering connection**. Fill in the peering name, a local VPC to peer with (VPC1), and another VPC to peer with (in my account and this region, it is VPC2). Then, click **Create peering connection**, as shown in *Figure 3.17*:

Create peering connection

A VPC peering connection is a networking connection between two VPCs that enables you to route traffic between them privately. [Info](#)

Peering connection settings

Name - optional
Create a tag with a key of 'Name' and a value that you specify.

VPC12

Select a local VPC to peer with

VPC ID (Requester)

vpc-0e6043d6091faa7cd (VPC1) ▼

VPC CIDRs for vpc-0e6043d6091faa7cd (VPC1)

CIDR	Status	Status reason
10.10.0.0/16	✔ Associated	-

Select another VPC to peer with

Account

☒ My account
☐ Another account

Region

☒ This Region (us-west-2)
☐ Another Region

VPC ID (Accepter)

vpc-05533af62e30e2168 (VPC2) ▼

VPC CIDRs for vpc-05533af62e30e2168 (VPC2)

CIDR	Status	Status reason
8.8.0.0/16	✔ Associated	-

Tags

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key

Q Name X

Value - optional

Q VPC12 X

Remove

Add new tag

You can add 49 more tags.

Cancel Create peering connection

Figure 3.17 – VPC peering

The connection is now in the *pending status*. Click it and accept the peering request, then it will be in the active state. The bridge is now built, but we still need to configure the routes between VPC1 and VPC2 – that is, configure route tables on both VPCs.

Go to the VPC dashboard and then to **Route tables**, select the VPC1 route table, **Edit routes**, then add a route from VPC1 to VPC2 (8.8.0.0/16) with the target of the peering connection (VPC12). Click **Save changes**. This is shown in *Figure 3.18*:

Destination	Target	Status	Propagated
10.10.0.0/16	Q local X	Active	No
Q 0.0.0.0/0 X	Q igw-02afe3e9778325c7e X	Active	No
Q 8.8.0.0/16 X	Q pcx-0b0a54adac1959d62 X	-	No

Add route

Cancel Preview Save changes

Figure 3.18 – Edit routes for the VPC1 route table

Repeat the preceding steps for the VPC2 route table – add a route from VPC2 to VPC1 (10.10.0.0/16) with the **peering connection** (VPC12) as the target. Then, associate it to the subnet (8.8.8.0/24).

There is one more thing to do. Go to the EC2 dashboard and then to **EC2-8 | Security | Security Group | Edit inbound rules**, and add a rule to EC-8's security group to allow ping from VPC1. Then, click **Save rules**, as shown in *Figure 3.19*:

Security group rule ID	Type	Protocol	Port range	Source	Description - optional
sgr-0851c34715be052b9	SSH	TCP	22	Custom	76.187.193.217/32
All ICMP - IPv4	All ICMP - IPv4	ICMP	All	Custom	10.10.0.0/16

Add rule

Cancel Preview changes Save rules

Figure 3.19 – Edit SG inbound rules for EC2-8

Now, let's verify that EC2-1 and EC2-8 can communicate with each other. Go back to EC2-1's guest OS terminal and try to ping EC2-8 (the IP address is 8.8.8.43). We can see it is working, as in *Figure 3.20*:

```
[ec2-user@ip-10-10-1-225 ~]$ ping 8.8.8.43
PING 8.8.8.43 (8.8.8.43) 56(84) bytes of data.
64 bytes from 8.8.8.43: icmp_seq=1 ttl=255 time=0.955 ms
64 bytes from 8.8.8.43: icmp_seq=2 ttl=255 time=0.553 ms
64 bytes from 8.8.8.43: icmp_seq=3 ttl=255 time=0.495 ms
^C
--- 8.8.8.43 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2023ms
rtt min/avg/max/mdev = 0.495/0.667/0.955/0.206 ms
```

Figure 3.20 – Ping EC2-8 from EC2-1

Figure 3.21 shows the AWS VPC network we have built so far:

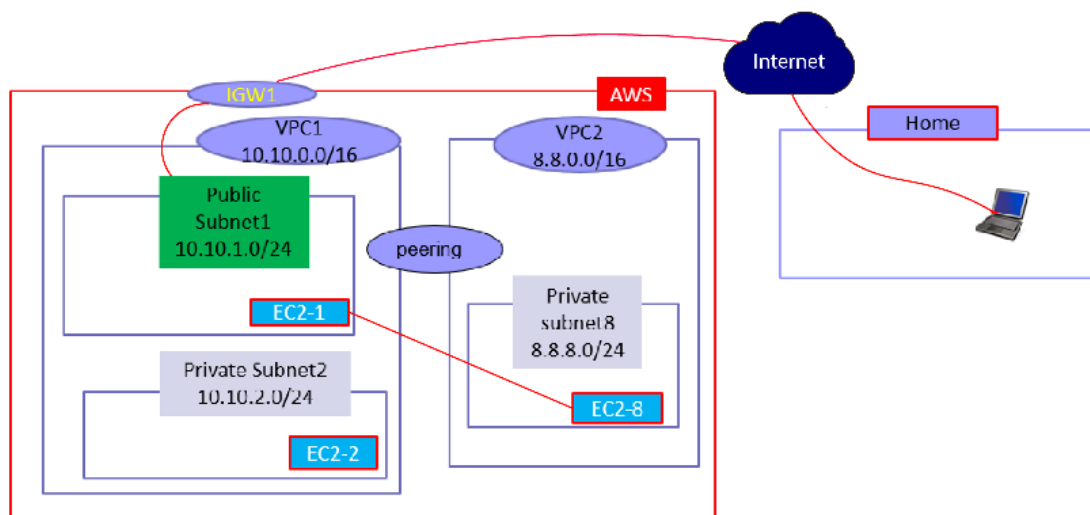


Figure 3.21 – VPC network with peering

Now, we have opened communication channels between EC2-1 in the public subnet and the internet via IGW1. But what about the EC2-2 and EC2-8 instances that are in the private subnets of VPC1 and VPC2? In the next section, we will discuss a way for EC2-2 and EC2-8, the two instances that do not have public IP addresses, to access the internet.

Creating a Network Address Translation (NAT) gateway

A NAT gateway allows EC2 instances in a VPC to go outbound to resources on the internet, but does not allow inbound traffic to the EC2 instance from the internet. Different from the IGW way in which we assign public IP addresses to EC2, here we use NAT to map EC2's private IP addresses to the NAT's public address for outbound requests and map the public IP address back to the EC2 private IP addresses for inbound responses. Take the following steps:

1. **Create a public NAT gateway.** Go to the VPC dashboard and then to **NAT gateways** | **Create NAT gateway**. Fill in the name (NAT1) and subnet (public subnet) and click **Allocate Elastic IP** to get an **Elastic IP (EIP)** for NAT1, then click **Create NAT gateway**, as shown in *Figure 3.22*.

Create NAT gateway [Info](#)

A highly available, managed Network Address Translation (NAT) service that instances in private subnets can use to connect to services in other VPCs, on-premises networks, or the internet.

NAT gateway settings

Name - optional
Create a tag with a key of 'Name' and a value that you specify.
NAT1
The name can be up to 256 characters long.

Subnet
Select a subnet in which to create the NAT gateway.
subnet-086224ff1a4e9f46f (pn)

Connectivity type
Select a connectivity type for the NAT gateway.
☒ Public
☐ Private

Elastic IP allocation ID [Info](#)
Assign an Elastic IP address to the NAT gateway.
eipalloc-014fb2aff49fc11a6 [Allocate Elastic IP](#)

[Additional settings](#) [Info](#)

Tags
A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key **Value - optional**

Q Name X Q NAT1 X Remove

[Add new tag](#)
You can add 49 more tags.

[Cancel](#) [Create NAT gateway](#)

Figure 3.22 – Create NAT gateway

2. Set up the private subnet route table:

After the NAT gateway is created, navigate to the VPC dashboard and then **route tables** and create a new route table called `VPC1PriRT` under `VPC1`. Add a route under **Destination** (`0.0.0.0/0`) using `NAT1` we created earlier as the target, then click **Save changes**, as shown in *Figure 3.23*. Then, associate the route table to the private subnet.

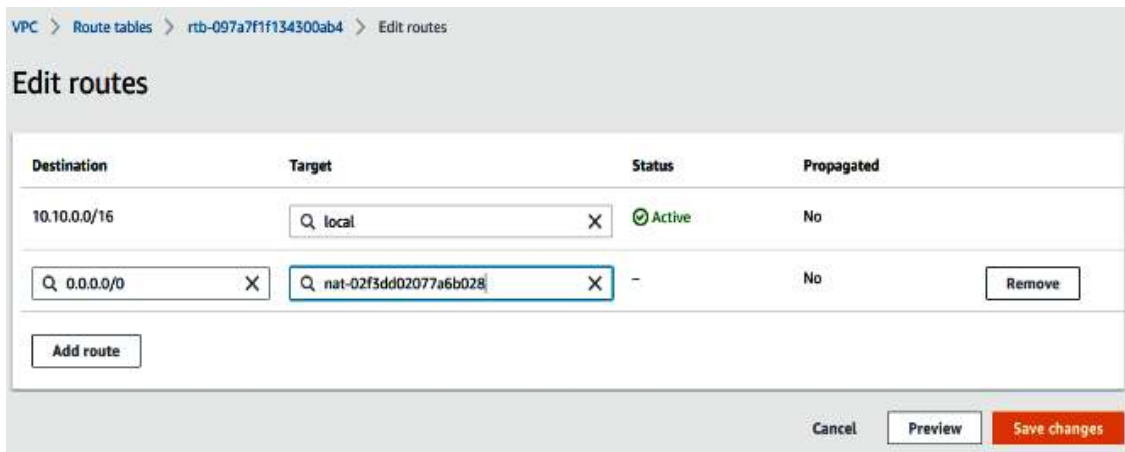


Figure 3.23 – Edit routes for the private subnet route table

3. Validate internet access from EC2-2

Now, SSH to EC2-1 (via the IGW), then SSH from EC2-1 to EC2-2 (make sure the SG for EC2-2 allows SSH traffic from EC2-1, and the private key is copied to EC2-1) and execute the `curl www.google.com` command. As shown in *Figure 3.24*, it works (the curl messages are partially shown)!

```
[ec2-user@ip-10-10-1-229 ~]$ ls -l
total 4
-r----- 1 ec2-user ec2-user 1675 Feb 15 16:10 mykey.pem
[ec2-user@ip-10-10-1-229 ~]$ ssh 10.10.2.249 -i mykey.pem
The authenticity of host '10.10.2.249 (10.10.2.249)' can't be established.
ECDSA key fingerprint is SHA256:AYodQL9fBbvlN6XRQLbHSjkNtbsFGevS7CqHSmr2vrk.
ECDSA key fingerprint is MD5:77:e0:ec:a4:40:61:95:68:c3:94:ba:62:98:23:35:7f.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added '10.10.2.249' (ECDSA) to the list of known hosts.

  _ | _ | _ )
  _ | ( _ /  Amazon Linux 2 AMI
  _ | \ _ | _ |

https://aws.amazon.com/amazon-linux-2/
[ec2-user@ip-10-10-2-249 ~]$ curl www.google.com
<!doctype html><html itemscope="" itemtype="http://schema.org/WebPage" lang="en"
><head><meta content="Search the world's information, including webpages, images
, videos and more. Google has many special features to help you find exactly wha
```

Figure 3.24 – Access `www.google.com` from EC2-2

Now, we have implemented the AWS cloud architecture shown in *Figure 3.25*, and successfully connected EC2-1 to the internet via IGW1 and EC2-2 via NAT1-IGW1:

AWS VPC Architecture

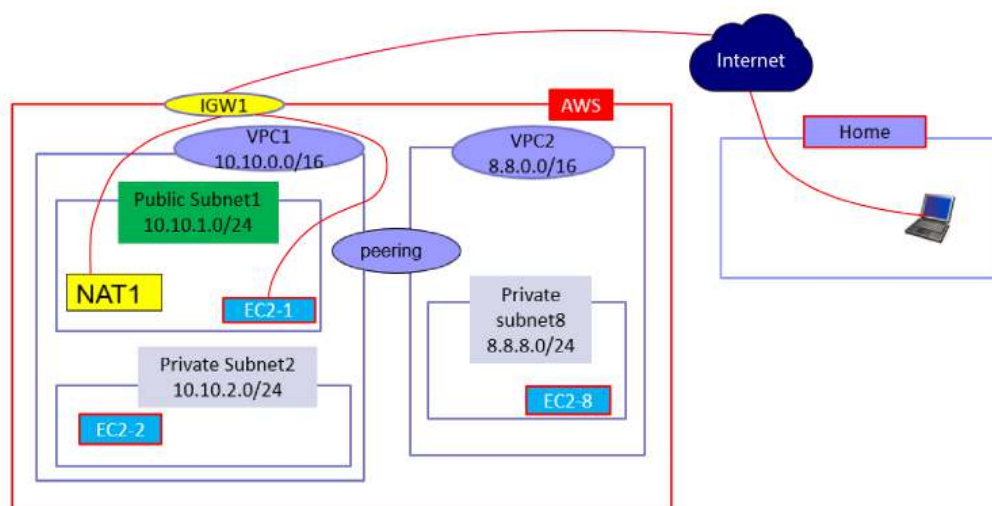


Figure 3.25 – AWS VPC architecture

As a practice lab, configure subnet8 to allow EC2-8 outbound access to the internet (but not inbound from the internet).

Part three – hardening AWS network security

Since we opened VPC1 to the internet, security has become a very important factor. Here, we will discuss the configurations in VPC to harden the security of the VPC and the cloud resources sitting in it. We will discuss two topics: VPC firewalls and VPC endpoints.

VPC firewalls

From *Chapter 1*, we know that a key pair and security groups are used to protect EC2 instances. Here, we will review **SGs**, which are virtual firewalls for EC2 instances and have the following features:

- Act as a virtual firewall that controls the inbound and outbound traffic for one or more EC2 instances
- Can be used to allow or deny traffic based on IP address, port number, and protocol (such as TCP or UDP)
- Can be associated with one or more EC2 instances or with a specific network interface of an instance

- Support the principle of least privilege, which means that only necessary traffic is allowed to reduce the attack surface and improve security
- Are stateful, which means that any traffic that is allowed to come into the EC2 instances will be allowed to go out; any traffic that is allowed to go out will be allowed to come in

Let me use an example to explain the term *stateful*. As shown in *Figure 3.26*, you have a PC at home with the IP address 54 . 24 . 12 . 19 that needs to SSH to an EC2 instance in the Amazon cloud with the address 10 . 10 . 1 . 1. Since the SG has an inbound rule open for SSH at port 22 from 54 . 24 . 12 . 19, the packet (source IP: 54 . 24 . 12 . 19, source port: 2000, destination IP: 10 . 10 . 1 . 1, and destination port: 22) is allowed to go into the EC2 server. When the packet is received by the EC2 instance, it will compile a return packet (source IP: 10 . 10 . 1 . 1, source port: 22, destination IP: 54 . 24 . 12 . 19, and destination port: 2000). The SG is stateful, meaning that the return packet will be allowed to go out, and there is no need to add any outbound rules for the packet:

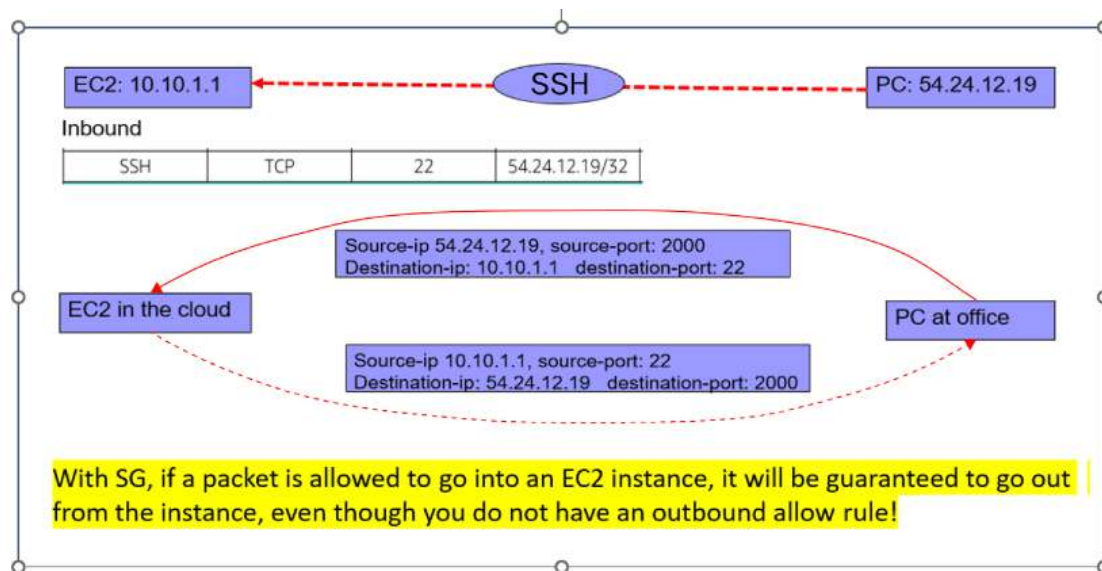


Figure 3.26 – SG is stateful (1)

Figure 3.27 shows another SG example for outbound traffic. You have an EC2 instance in the Amazon cloud with the IP address 10 . 10 . 1 . 1 and need to access a web server running on an office PC with the address 54 . 24 . 12 . 19. Since the SG has an outbound rule open for port 80 from 10 . 10 . 1 . 1, the packet (source IP: 10 . 10 . 1 . 1, source port: 1234, destination IP: 54 . 24 . 12 . 19, and destination port: 80) will be allowed to go out to the PC server. When the packet is received by the web server, it will compile a return packet (source IP: 54 . 24 . 12 . 19, source port: 80, destination IP: 10 . 10 . 1 . 1, and destination port: 1234). The SG is stateful, meaning that the return packet will be allowed to come in, and there is no need to add any inbound rules for the packet:

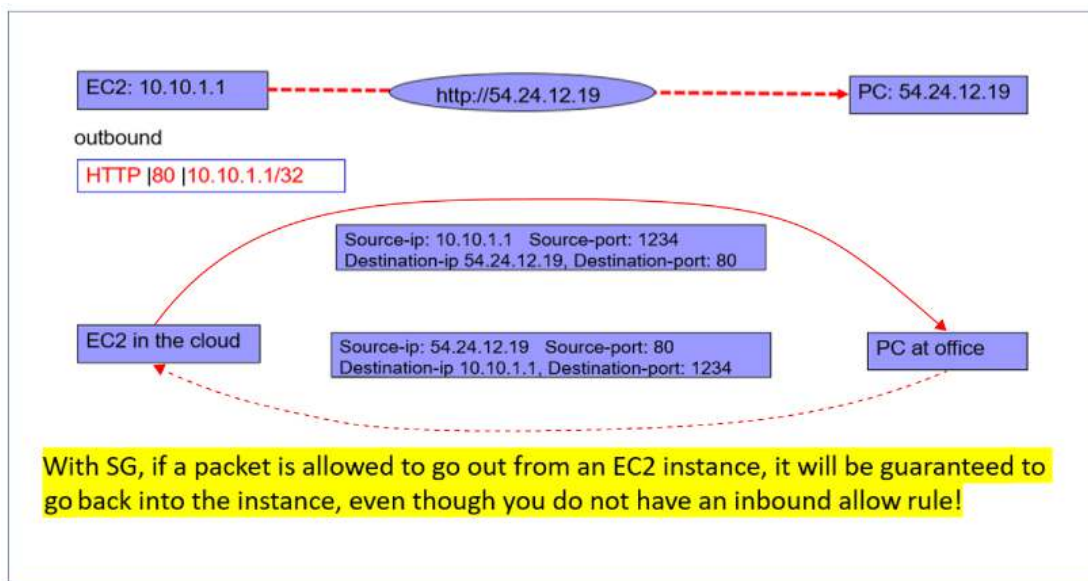


Figure 3.27 – SG is stateful (2)

An NACL is another type of firewall at the subnet level, and it has the following features:

- It is used to allow or deny traffic based on the IP address, port number, and protocol
- It is evaluated before SGs and applies to all instances in a subnet, whereas security groups are associated with individual instances
- It supports numbered rules that are evaluated in order, with the lowest number taking precedence
- It is stateless, which means that any traffic that is allowed to go into the EC2 instances is not guaranteed to be allowed to go out, and we need an NACL rule to explicitly specify the outbound permission

Let us use an example to explain the term *stateless*. As shown in Figure 3.28, you have a PC at home with the IP address 54 . 24 . 12 . 19 and need to SSH to an EC2 instance in the Amazon cloud with the address 10 . 10 . 1 . 1. Since the NACL has an inbound rule to allow for SSH traffic at port 22 from 54 . 24 . 12 . 19, the packet (source IP: 54 . 24 . 12 . 19, source port: 2000, destination IP: 10 . 10 . 1 . 1, and destination port: 22) is allowed to go into VPC1. When the packet is received by the EC2 instance, it will compile a return packet (source IP: 10 . 10 . 1 . 1, source port: 22, destination IP: 54 . 24 . 12 . 19, and destination port: 2000). Because NACLs are stateless, the return packet may not be allowed to go out, and you will need to add an outbound rule for the packet:

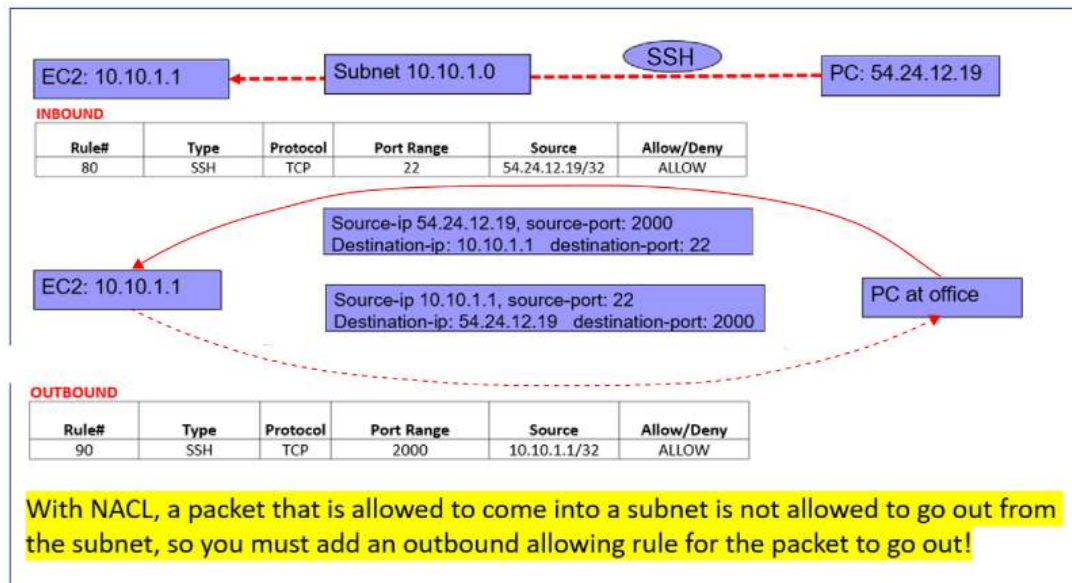


Figure 3.28 – NACL is stateless (1)

Figure 3.29 shows another example of outbound traffic. You have an EC2 instance in the Amazon cloud with the IP address 10.10.1.1 and need to access a web server running on a PC with the address 54.24.12.19. Since the NACL has an outbound rule to allow traffic to go out from port 80 from 10.10.1.1, the packet (source IP: 10.10.1.1, source port: 1234, destination IP: 54.24.12.19, and destination port: 80) will be allowed to go out to the PC server. When the packet is received by the web server running on the PC, it will compile a return packet (source IP: 54.24.12.19, source port: 80, destination IP: 10.10.1.1, and destination port: 1234). The NACL is stateless, meaning that the return packet may not be allowed to come in, unless there is an explicit inbound rule added to allow the packet to come back in:

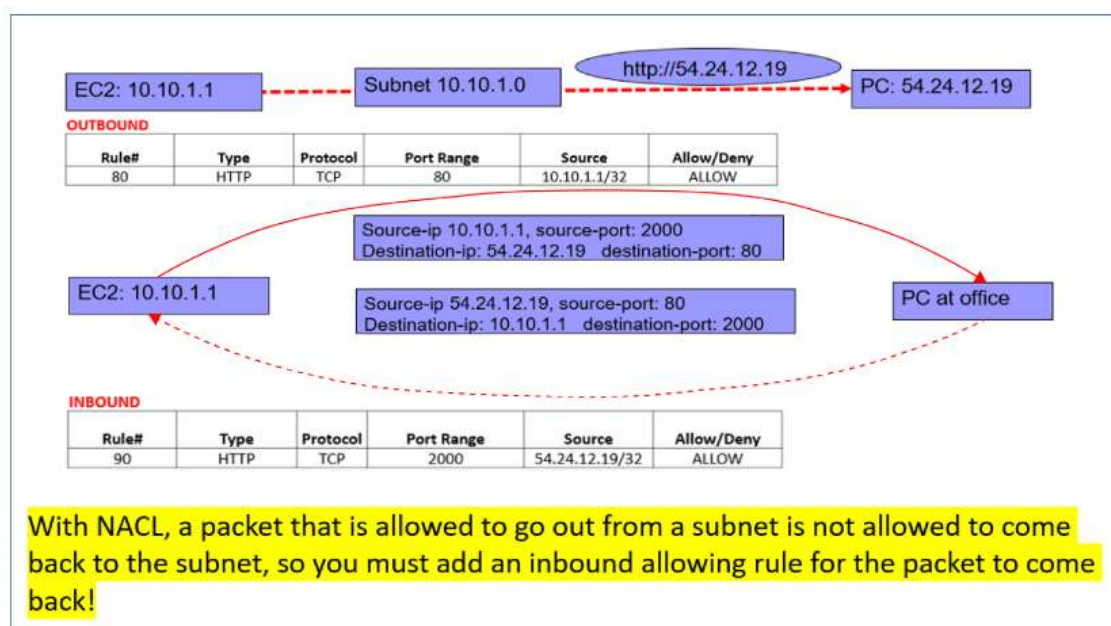


Figure 3.29 – NACL is stateless (2)

As we have learned from the preceding discussions, SGs and NACLs provide protection to the cloud resources inside a VPC, such as EC2 instances. But what about EC2 instances in a VPC to access an S3 bucket? We will introduce VPC endpoints for S3 next.

VPC endpoints

In the previous sections, we learned that an IGW provides inbound and outbound communication with internet resources, and NAT only provides outbound internet access for private cloud resources. Any EC2 instances that have internet access can access public endpoints such as S3 buckets after being assigned an EC2 role that has permission to access the buckets. However, due to security reasons, some companies do not allow internet trespassing to access AWS public endpoints from VPC resources. To that end, VPC endpoints were introduced.

A VPC endpoint for S3 is a way to access S3 resources privately within an Amazon VPC without using a public IGW, a VPN connection, or NAT devices. Using the network diagram shown in *Figure 3.30*, we will create a VPC endpoint for S3 and provide S3 bucket access for an EC2-2 instance that has no public IP addresses:

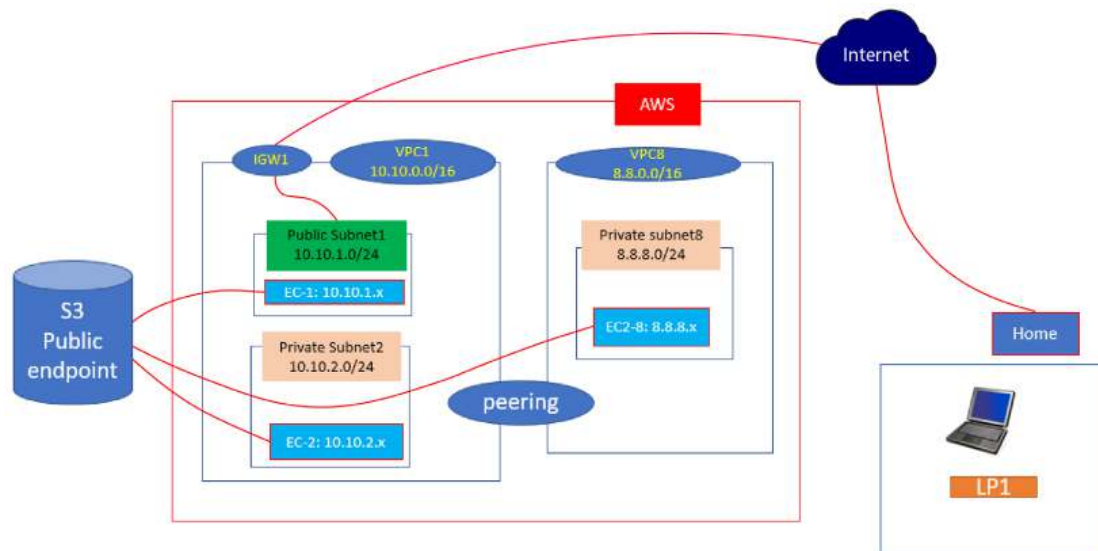


Figure 3.30 – VPC network diagram

Here are the detailed steps:

First, we need to create an EC2 role that has access to the S3 bucket and assign the role to our EC2 instance. We discussed and did this in the previous chapter, and now we create the endpoint:

1. Create a VPC endpoint for S3 in the us-east1 region:
 - I. Log in to the AWS Management Console and navigate to the Amazon VPC console. Click on **Endpoints** in the left-hand menu. Click the **Create Endpoint** button.
 - II. Fill in the endpoint settings in the new pop-up window, as shown in *Figure 3.31*:
 - vpc-s3-endpoint is the name
 - Select **AWS services** as the service category
 - Select **com.amazonaws.us-east-1.s3** as the service name
 - Select **VPC1** to create the endpoint
 - Select the **VPC1-Pri** route table to associate with the endpoint
 - Enable policies for resource-based policies to control access to the bucket
 - III. Click **Create endpoint** to create the VPC endpoint.
2. Now you can verify S3 access from EC2-2, which does not have internet access.

Create endpoint [Info](#)

There are three types of VPC endpoints – Interface endpoints, Gateway Load Balancer endpoints, and Gateway endpoints. Interface endpoints and Gateway Load Balancer endpoints are powered by AWS PrivateLink, and use an Elastic Network Interface (ENI) as an entry point for traffic destined to the service. Interface endpoints are typically accessed using the public or private DNS name associated with the service, while Gateway endpoints and Gateway Load Balancer endpoints serve as a target for a route in your route table for traffic destined for the service.

Endpoint settings

Name tag - optional
Creates a tag with a key of 'Name' and a value that you specify.

Service category
Select the service category

☒ **AWS services**
Services provided by Amazon

☐ **PrivateLink Ready partner services**
Services with an AWS Service Ready designation

☐ **AWS Marketplace services**
Services that you've purchased through AWS Marketplace

☐ **Other endpoint services**
Find services shared with you by service name

Services (1/3)

☒ **Service Name:** com.amazonaws.us-east-1.s3

	Service Name	Owner	Type
☒	com.amazonaws.us-east-1.s3	amazon	Gateway
☐	com.amazonaws.us-east-1.s3	amazon	Interface
☐	com.amazonaws.us-east-1.s3-outposts	amazon	Interface

VPC

Select the VPC in which to create the endpoint

VPC
The VPC in which to create your endpoint.

Route tables (1/2) [Info](#)

	Name	Route Table ID	Main
☐	VPC1RT	rtb-0595676d643322bb5 (VPC1RT)	Yes
☒	VPC1-Pri	rtb-0fa2616ffbf955b3c (VPC1-Pri)	No

Figure 3.31 – VPC endpoint settings